

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12401686
Kind Code	B2
Date of Patent	August 26, 2025
Inventor(s)	El-Moussa; Fadi et al.

Detecting changes to web page characteristics using machine learning

Abstract

A computer implemented method to detect an anomalous change to a web application, the web application executing with a web server, the method including receiving a first set of records for the web application operating in a training mode of operation, each record including characteristics of a content of a web page for the web application; generating a sparse distributed representation of the set of records to form a training set for a hierarchical temporal memory (HTM); training the HTM based on the training set in order that the trained HTM provides a model of the operation of the web application in the training mode of operation; receiving a second set of records for the web application, each record including characteristics of content of the web page; generating a sparse distributed representation of the second set of records to form an input set for the trained HTM; executing the trained HTM based on the input set to determine a degree of recognition of the records of the input set; and responsive to a determination that a degree of recognition of one or more records of the input set is below a threshold degree, identifying an anomalous change to the web page.

Inventors:	El-Moussa; Fadi (London, GB), Du; Xiaofeng (London, GB)
Applicant:	BRITISH TELECOMMUNICATIONS PUBLIC LIMITED COMPANY (London, GB)
Family ID:	1000008782475
Assignee:	British Telecommunications Public Limited Company (London, GB)
Appl. No.:	16/636282
Filed (or PCT Filed):	July 30, 2018
PCT No.:	PCT/EP2018/070632
PCT Pub. No.:	WO2019/025386
PCT Pub. Date:	February 07, 2019

Prior Publication Data

Document Identifier	Publication Date
US 20200228569 A1	Jul. 16, 2020

Foreign Application Priority Data

EP	17184578	Aug. 02, 2017
----	----------	---------------

Publication Classification

Int. Cl.: H04L9/00 (20220101); G06F16/958 (20190101); G06N3/045 (20230101); H04L9/40 (20220101)

U.S. Cl.:

CPC H04L63/1483 (20130101); G06F16/958 (20190101); G06N3/045 (20230101); H04L63/1416 (20130101);

Field of Classification Search

CPC: H04L (63/1433); G06K (9/6256)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
10769292	12/2019	Daniel	N/A	N/A
10951651	12/2020	Golan et al.	N/A	N/A
11050768	12/2020	Stickle	N/A	H04L 63/1425
2009/0282480	12/2008	Lee	717/130	G06F 21/55
2009/0313193	12/2008	Hawkins et al.	N/A	N/A
2012/0005753	12/2011	Provos et al.	N/A	N/A
2012/0047581	12/2011	Banerjee et al.	N/A	N/A
2013/0054496	12/2012	Marianetti, II et al.	N/A	N/A
2014/0067734	12/2013	Hawkins	706/58	G06N 7/01
2015/0127595	12/2014	Hawkins, II	706/46	G06N 7/005
2016/0028762	12/2015	Di Pietro et al.	N/A	N/A
2016/0321557	12/2015	Hawkins et al.	N/A	N/A
2017/0053025	12/2016	De Sousa Webber	N/A	G06F 16/358
2017/0068816	12/2016	Cavazos	N/A	N/A
2017/0104773	12/2016	Flacher	N/A	H04L 63/1458
2017/0116412	12/2016	Stopel et al.	N/A	N/A
2017/0116415	12/2016	Stopel et al.	N/A	N/A
2017/0126718	12/2016	Baradaran	N/A	H04L 63/0281
2018/0020024	12/2017	Chao	N/A	G06F 17/18
2018/0124080	12/2017	Christodorescu	N/A	G06F 21/552
2021/0089650	12/2020	Daniel	N/A	N/A
2021/0089670	12/2020	Daniel	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
102411687	12/2011	CN	N/A
106951783	12/2016	CN	N/A
1814055	12/2006	EP	N/A
WO-2017194637	12/2016	WO	N/A

OTHER PUBLICATIONS

Ahmad S., et al., “How do Neurons Operate on Sparse Distributed Representations? A Mathematical Theory of Sparsity, Neurons and Active Dendrites,” Retrieved from <https://arxiv.org/ftp/arxiv/papers/1601/1601.00720.pdf>, 2018, 23 pages. cited by applicant

Ahmad S., et al., “Properties of Sparse Distributed Representations and their Application to Hierarchical Temporal Memory,” retrieved from <https://arxiv.org/ftp/arxiv/papers/1503/1503.07469.pdf> on Mar. 28, 2018, Numenta, Mar. 24, 2015, 18 pages. cited by applicant

Ahmad S., et al., “Real-Time Anomaly Detection for Streaming Analytics,” retrieved from <https://arxiv.org/pdf/1607.02480.pdf> on Mar. 28, 2018, Numenta, Inc., Jul. 8, 2016, 10 pages. cited by applicant

Antonopoulos A.M., “Mastering Bitcoin, Unlocking Digital Crypto-Currencies,” O'Reilly Media, Apr. 2014, 282 pages. cited by applicant

Assia Y., et al., “Colored Coins Whitepaper,” 2015, available at https://docs.google.com/document/d/1AnkP_cVZTCMLIzw4DvsW6M8Q2JC0llzrTLuoWu2z1BE/, 23 pages. cited by applicant

Berger V., “Anomaly Detection in User Behavior of Websites Using Hierarchical Temporal Memories,” May 19, 2017, retrieved from URL <http://kth.diva-portal.org/smash/get/diva2:1094877/FULLTEXT01.pdf> on Sep. 5, 2017. cited by applicant

Billaudelle S., et al., “Porting HTM Models to the Heidelberg Neuromorphic Computing Platform,” Feb. 9, 2016, Cornell University Library, retrieved from <https://arxiv.org/pdf/1505.02142.pdf>, 9 pages. cited by applicant

Communication pursuant to Article 94(3) EPC for European Application No. 18811033.2, mailed Jan. 14, 2022, 4 pages. cited by applicant

Cui Y., et al., “Continuous Online Sequence Learning with an Unsupervised Neural Network Model,” Neural Computation, vol. 28, No. 11, Nov. 2016, pp. 2474-2504. cited by applicant

Cui Y., et al., “The HTM Spatial Pooler: A Neocortical Algorithm for Online Sparse Distributed Coding,” retrieved from <https://www.biorxiv.org/content/biorxiv/early/2017/02/16/085035.full.pdf> on Mar. 28, 2018, Numenta Inc., Feb. 3, 2017, 16 pages. cited by applicant

Hawkins J., et al., “Hierarchical Temporal Memory Concepts, Theory, and Terminology,” Numenta Inc., 2006, 19 pages. cited by applicant

Hawkins J., et al., “Why Neurons Have Thousands of Synapses, A Theory of Sequence Memory in Neocortex,” Frontiers in Neural Circuits, vol. 10, Article 23, Mar. 2016, 13 pages. cited by applicant

Hawkins J., “On Intelligence,” How a New Understanding of the Brain Will Lead to the Creation of Truly Intelligent Machines,

2004, Times Books, Jul. 14, 2005, 174 pages. cited by applicant
International Preliminary Report on Patentability for Application No. PCT/EP2018/070631, mailed on Feb. 13, 2020, 9 pages. cited by applicant
International Preliminary Report on Patentability for Application No. PCT/EP2018/070632, mailed on Feb. 13, 2020, 10 pages. cited by applicant
International Search Report and Written Opinion for Application No. PCT/EP2018/070631, mailed on Aug. 17, 2018, 12 pages. cited by applicant
International Search Report and Written Opinion for Application No. PCT/EP2018/070632, mailed on Aug. 21, 2018, 13 pages. cited by applicant
International Search Report and Written Opinion For PCT Application No. PCT/EP2018/083358, mailed on Feb. 15, 2019, 11 pages. cited by applicant
Lavin A., et al., "Evaluating Real-Time Anomaly Detection Algorithms—The Numenta Anomaly Benchmark," Retrieved from <https://arxiv.org/ftp/arxiv/papers/1510/1510.03336.pdf>, Numenta, Inc., Oct. 9, 2015, 8 pages. cited by applicant
Network Working Group, "Hypertext Transfer Protocol—HTTP/1.1," Jun. 1999, 114 Pages. cited by applicant
Numenta, "Biological and Machine Intelligence (BAMI), A living book that documents Hierarchical Temporal Memory (HTM)," Mar. 8, 2017, 69 pages. cited by applicant
Numenta, "Hierarchical Temporal Memory including HTM Cortical Learning Algorithms," Version 0.2.1, Numenta, Sep. 12, 2011, 68 pages. cited by applicant
Numenta, "The Science of Anomaly Detection," How HTM Enables Anomaly Detection in Streaming Data, Dec. 31, 2015, Retrieved from URL:<https://numenta.com/assets/pdf/whitepapers/Numenta%20White%20Paper%20-%20Science%20of%20Anomaly%20Detection.pdf> retrieved on Sep. 7, 2017. cited by applicant
Office Action For GB Application No. 17184578.7, mailed on Sep. 18, 2017, 11 pages. cited by applicant
Office Action For GB Application No. 17184580.3, mailed on Nov. 3, 2017, 8 pages. cited by applicant
Office Action For GB Application No. 1720174.0, mailed on May 23, 2018, 6 pages. cited by applicant
Olshausen B.A., et al., "Sparse Coding with an Overcomplete Basis Set: A Strategy Employed by VI?," Pergamon, vol. 37, No. 23, 1997, pp. 3311-3325. cited by applicant
Purdy S., "Encoding Data for HTM Systems," Retrieved from <https://arxiv.org/ftp/arxiv/papers/1602/1602.05925.pdf>, Numenta, Inc., Feb. 2016, 11 pages. cited by applicant
Rosenfeld M., "Overview of Colored Coins," <https://bravenewcoin.com/assets/Whitepapers/Overview-of-Colored-Coins.pdf>, Dec. 4, 2012, 13 pages. cited by applicant
Taylor M., "Sparse Distributed Representations," Numenta, 2017, 3 pages. cited by applicant
Thorpe S.J., "Spike Arrival Times: A Highly Efficient Coding Scheme for Neural Networks," Parallel Processing in Neural Systems and Computers, 1990, pp. 91-94. cited by applicant
Wang, C. et al., "A Distributed Anomaly Detection System for In-Vehicle Network Using HTM," 2018, vol. 6, pp. 9091-9098. cited by applicant
Wood G., "Ethereum: A Secure Decentralised Generalized Transaction Ledger," EIP-150 Revision, Jun. 4, 2014, pp. 1-32. cited by applicant
Application and File History for U.S. Appl. No. 16/636,280, filed Feb. 3, 2020, Inventors: El Moussa et al. cited by applicant
Application and File History for U.S. Appl. No. 15/733,180, filed Jun. 4, 2020, Inventors: El Moussa et al. cited by applicant

Primary Examiner: Brown; Christopher J

Attorney, Agent or Firm: EIP US LLP

Background/Summary

PRIORITY CLAIM

(1) The present application is a National Phase entry of PCT Application No. PCT/EP2018/070632, filed Jul. 30, 2018, which claims priority from EP Patent Application No. 17184578.7, filed Aug. 2, 2017, which is hereby fully incorporated herein by reference.

TECHNICAL FIELD

(2) The present disclosure relates to the detection of changes to web page characteristics of web applications.

BACKGROUND

(3) Malicious attacks to network connected computer systems present a serious and growing concern for the security of the systems, data stored and/or communicated by such systems and other computing entities operating in communication with such systems. For example, a server component such as a web server can be compromised, such as by exploitation of a software or network vulnerability. Information, configuration and communications involving such compromised components are then at risk of potential disclosure to malicious entities. Additionally or alternatively, such a compromised system can be used to communicate with other malicious entities including other network connected components that are vulnerable to exploitation, such as client computer systems. Such attacks can include, for example, cross site scripting (XSS) and malware infection of a web server.

SUMMARY

(4) Preventing such malicious attacks requires first the detection of such attacks and it would be desirable to provide techniques

for detecting malicious attacks in order that malicious entities can be constrained in their effort to spread malicious software throughout a network.

(5) The present disclosure accordingly provides, in a first aspect, a computer implemented method to detect an anomalous change to a web application, the web application executing with a web server, the method comprising: receiving a first set of records for the web application operating in a training mode of operation, each record including characteristics of a content of a web page for the web application; generating a sparse distributed representation of the set of records to form a training set for a hierarchical temporal memory (HTM); training the HTM based on the training set in order that the trained HTM provides a model of the operation of the web application in the training mode of operation; receiving a second set of records for the web application, each record including characteristics of content of the web page; generating a sparse distributed representation of the second set of records to form an input set for the trained HTM; executing the trained HTM based on the input set to determine a degree of recognition of the records of the input set; and responsive to a determination that a degree of recognition of one or more records of the input set is below a threshold degree, identifying an anomalous change to the web page.

(6) In some embodiments, in response to the identification of the anomalous change to the web page, implementing a responsive measure to the anomalous change.

(7) In some embodiments, the responsive measure includes one or more of: interrupting operation of the web application; identifying client components in communication with the web application as potentially compromised; executing an intrusion detection, malware detection, virus removal and/or malware removal process for the web application; and effecting a redeployment, reinstallation and/or reconfiguration of the web application.

(8) In some embodiments, in the training mode of operation, the HTM evaluates an anomaly score for records in the first set of records and the HTM is trained until the anomaly score meets a predetermined threshold degree of anomaly.

(9) In some embodiments, the characteristics of the web page include records corresponding to hypertext markup language (HTML) tags in the web page.

(10) The present disclosure accordingly provides, in a second aspect, a computer system including a processor and memory storing computer program code for performing the method set out above.

(11) The present disclosure accordingly provides, in a third aspect, a computer program element comprising computer program code to, when loaded into a computer system and executed thereon, cause the computer to perform the method set out above.

Description

BRIEF DESCRIPTION OF THE FIGURES

(1) Embodiments of the present disclosure will now be described, by way of example only, with reference to the accompanying drawings, in which:

(2) FIG. 1 is a block diagram a computer system suitable for the operation of embodiments of the present disclosure.

(3) FIG. 2 is an illustration of an exemplary arrangement of a computer system configured to execute a web application according to embodiments of the present disclosure.

(4) FIG. 3 is a component diagram illustrating the operation of a web application change detector component to train a hierarchical temporal memory in accordance with embodiments of the present disclosure.

(5) FIG. 4 is a component diagram illustrating the operation of the web application change detector component to detect a change for a web application in accordance with embodiments of the present disclosure.

(6) FIG. 5 is a flowchart of a method to detect an anomalous change to a web application executing with a web server in accordance with embodiments of the present disclosure.

DETAILED DESCRIPTION OF EMBODIMENTS

(7) FIG. 1 is a block diagram of a computer system suitable for the operation of embodiments of the present disclosure. A central processor unit (CPU) **102** is communicatively connected to a storage **104** and an input/output (I/O) interface **106** via a data bus **108**. The storage **104** can be any read/write storage device such as a random access memory (RAM) or a non-volatile storage device. An example of a non-volatile storage device includes a disk or tape storage device. The I/O interface **106** is an interface to devices for the input or output of data, or for both input and output of data. Examples of I/O devices connectable to I/O interface **106** include a keyboard, a mouse, a display (such as a monitor) and a network connection.

(8) Malicious attacks on web servers frequently lead to a change to configuration of a web server that can exhibit as changes to characteristics of a web application including subtle changes. Such changes can include, inter alia, changes to HTTP headers, HTTP response messages, traffic management changes, connection closure mechanics and changes to characteristics of web pages themselves. Embodiments of the present invention seek to detect anomalous changes to a web application configuration or changes to a web application itself being indicative of malicious intervention.

(9) FIG. 2 is an illustration of an exemplary arrangement of a computer system **200** configured to execute a web application **298** according to embodiments of the present disclosure. The computer system **200** can be a physical, virtual or combination computer system communicatively connected with one or more clients **206**. The clients **206** and computer system **200** can be communicatively connected via one or more network connections **284, 285** such as wired or wireless network connections including connections via an intermediate network such as a wide area network or the internet **208**. The computer system includes a web server **296** as a hardware, software, firmware, or combination component for providing one or more web applications **298** as are well known in the art. In use, the clients **206** communicate with the server **296** via the network to access and utilize the web application **298**.

(10) The web server **296** includes a configuration comprised of a plurality of features of the web server **296** including, for example, inter alia: a web server type such as a particular web server software component characterized by its vendor, model, version or the like; web technology support characteristics such as which version of one or more web or other protocols are

supported by the web server **296**, a configuration of the web server **296** in relation to one or more other components operating in, with or in communication with the computer system **200** such as middleware components, data storage components like databases, enterprise software components and other components as will be apparent to those skilled in the art; one or more configurations of how the web server **296** undertakes traffic management including how web server messages, content and the like are divided or split between network communications; the nature of the web server header format for HTTP message headers; the parameters of the web server to be included in web server response messages; the particular web pages for the web application **298** including templates, cascading style sheet (CSS) definitions, scripts, libraries and the like; server-side scripting such as PHP scripts and runtime; and other configuration and application characteristics for the web application **298**.

(11) FIG. **3** is a component diagram illustrating the operation of a web application change detector component **202** to train a hierarchical temporal memory (HTM) **320** in accordance with embodiments of the present disclosure. A training client **306** is provided which is a network connected software, hardware, firmware or combination component suitable for communication with the computer system **200** to operate as a client of the web application **298**. For example, the client **306** includes a web browser suitable for accessing static, dynamic and/or rich internet web pages constituting a client portion and/or interface to or for the web application **298**. Accordingly, the training client **306** operates as a client of the web application **298** in a training mode of operation in which the computer system, web server, web application and training client are absent any effect, tampering, infection or other encumbrance by malicious software, services or communications such as malware or attacks. For example, preferably the training client **306** and computer system **200** operate in a segregated, isolated, localized or otherwise protected environment such as a local network being disconnected from a wide area network or internet such that malicious encumbrances are avoided during the training mode of operation. The training mode of operation is a mode in which a model of typical characteristics of the web application **298**, the web server **296**, the computer system **200** and/or one or more web pages provided by the web server **296** for the application **298** are modelled for subsequent use in anomaly detection. Thus, the training client **306** operates to access and/or use the web application **298** by accessing static and/or dynamic web pages of the application **298** in a normal usage absent malicious encumbrances. In a preferred embodiment, such normal usage by the training client **306** is arranged to cover at least a minimum proportion of a range of features, facilities, web pages, services or other aspects of the web application **298** in order that characteristics for the web application **298** are broadly modelled.

(12) Characteristics of the web application and/or web page(s) of the web application are collected during the training mode of operation. In one embodiment, the characteristics include at least part of a content of an HTTP header generated by the web server **296** for communications as part of the execution of the application **298**. Additionally or alternatively, the characteristics can include an indication, definition, identification or other characteristics relating to how the web server **296** manages network traffic communicated to the training client **306** such as a profile of how network traffic is split, divided or distributed across network messages such as web server response messages sent to a browser at the training client **306**. Additionally or alternatively the characteristics can include features of one or more HTTP response messages generated by the web server **296** in response to requests received from the training client **306**. Such features of response messages can include particular parameters of response message fields as defined, for example, in HTTP response message format in section 6 of RFC2616 (Fielding, R. et al, The Internet Society, 1999, available at www.ietf.org/rfc/rfc2616.txt). Additionally or alternatively, the characteristics can include features of how the web server **296** closes a web connection such as, for example, by way of deferring to an underlying transport protocol such as a “FIN” message in TCP/IP, or by reliance on a web browser to close the connection. Yet further additionally or alternatively, the characteristics can include characteristics of a content of one or more web pages for the web application **298** such as hypertext markup language (HTML) tags in one or more webpages.

(13) For example, a record collector component **201** as a hardware, software, firmware or combination component can be arranged to access, obtain, measure or otherwise determine the characteristics and collect the characteristics as a series of records constituting a first record set **310**. In one embodiment, the record collector **201** is configured to receive HTTP messages from and/or via the training client **306** from which characteristics can be determined, derived, measured or inferred. In an alternative embodiment, the record collector **201** can additionally and/or alternatively receive information such as HTTP messages from the web server directly, such as by way of a proxy for the training client **306** or the web server **296**. In further alternative embodiments the record collector **201** can be communicatively connected to the web server **296** such as via a network. In one embodiment the record collector **201** can be provided with and/or on either the training client **306** or the computer system **200** such as a software component of the training client **306** or computer system **200**. In one embodiment, the record collector **201** is a multi-part component being at least partly provided as a software component installed on one or more of: the computer system **200**; the training client **306**; and/or another computer system or component.

(14) The web application change detector **202** receives a first record set **310** from the record collector **201** in respect of operation of the training client **306** in the training mode of operation. The first record set **310** is a set of records for characteristics of the web application **298** in relation to operation of the web application **298** with the training client **306** for a defined period of time—known as a training time period.

(15) The change detector **202** uses the records in the record set **310** to constitute training data inputs for training a hierarchical temporal machine (HTM) **320**. The HTM **320** is a machine learning construct based on principles first described by Jeff Hawkins in “On Intelligence” (2004, Times Books, ISBN 0-8050-7456-2) and described in detail by Numenta in “Hierarchical Temporal Memory including HTM Cortical Learning Algorithms” (Numenta, 12 Sep. 2011). The principles of, implementation of and operation of HTM **320** are beyond the scope of this description and are nonetheless available to the skilled person through existing publications including the papers and books below, each and/or all of which are usable by a skilled person to implement the HTM **320** and other associated features for embodiments of the present invention: “Biological and Machine Intelligence (BAMI)—A living book that documents Hierarchical Temporal Memory (HTM)” (Numenta, Inc., Mar. 8, 2017) (retrieved Mar. 29, 2017) available from numenta.com/assets/pdf/biological-and-machine-intelligence/BAMI-Complete.pdf “Porting HTM Models to the Heidelberg Neuromorphic Computing Platform” (Billaudelle, S. & Ahmad, S., May 2015) available from Cornell University Library (citation arXiv:1505.02142) and arxiv.org/abs/1505.02142 “Encoding Data for HTM Systems” (Purdy, S.,

February 2016) available from Cornell University Library (citation arXiv:1602.05925) and arxiv.org/abs/1602.05925 “Properties of Sparse Distributed Representations and their Application To Hierarchical Temporal Memory” (Ahmad, S. & Hawkins, J., March 2015) available from Cornell University Library (citation arXiv:1503.07469) and arxiv.org/abs/1503.07469 “How Do Neurons Operate on Sparse Distributed Representations? A Mathematical Theory of Sparsity, Neurons and Active Dendrites” (Ahmad, S. & Hawkins, J., January 2016) available from Cornell University Library (citation arXiv:1601.00720) and arxiv.org/abs/1601.00720 “Real-Time Anomaly Detection for Streaming Analytics” (Ahmad, S. & Purdy, S., July 2016) available from Cornell University Library (citation arXiv:1607.02480) and arxiv.org/abs/1607.02480 “Evaluating Real-time Anomaly Detection Algorithms—the Numenta Anomaly Benchmark” (Lavin, A. & Ahmad, S., October 2015) available from Cornell University Library (citation arXiv: 1510.03336) and arxiv.org/abs/1510.03336 “The HTM Spatial Pooler: A Neocortical Algorithm for Online Sparse Distributed Coding” (Cui, Y., Ahmad, S. & Hawkins, J., February 2017) available from Cold Spring Harbor Laboratory bioRxiv (citation doi.org/10.1101/085035) and biorxiv.org/content/biorxiv/early/2017/02/16/085035.full.pdf “Continuous Online Sequence Learning with an Unsupervised Neural Network Model” (Cui, Y., Ahmad, S. & Hawkins, K., November 2016) published in Published in Neural Computation (November 2016, Vol 28. No. 11) and available from numenta.com/papers/continuous-online-sequence-learning-with-an-unsupervised-neural-network-model/ “Why Neurons Have Thousands of Synapses, A Theory of Sequence Memory in Neocortex” (Hawkins, J. & Ahmad, S., March 2016) published in Frontiers in Neural Circuits (10 (2016) 1-13, doi:10.3389/fncir.2016.00023) and available from numenta.com/papers/why-neurons-have-thousands-of-synapses-theory-of-sequence-memory-in-neocortex/

(16) At a very high level, in one embodiment, the HTM **320** is implementable logically as a hierarchy of functional nodes. The hierarchy of nodes in the HTM **320** are suitable for identifying coincidences in a temporal sequence of input patterns received at an input layer in the hierarchy, with interconnections between the layers permitting such identifications to take place also at each other level in the hierarchy. In addition to an identification of coincidences by nodes in the HTM **320**, temporal relationships between coincidences can also be identified. Thus, in a purely exemplary arrangement, a first set of similar patterns occurring before a second set of similar patterns can be resolved to a coincidence (of the first set) with a temporal relationship to a coincidence (of the second set). The coincidences and temporal relations learned at each of many levels in the hierarchical HTM **320** provide for subsequent recognition, by the HTM **320**, of a conforming temporal sequence of input patterns and non-conformant sequences. Thus, the HTM **320** can be said to operate in: a learning mode of operation in which coincidences and relationships between coincidences are learned by adaptation of the HTM **320** (and as described with reference to FIG. 3); and an inference mode of operation in which the HTM **320** is executed (by which it is meant that the HTM **320** is applied) to process one or more inputs to determine a degree of recognition of the inputs by the HTM **320** based on what has been learned by the HTM **320** (and as described with reference to FIG. 4). Recognition can be based on a determination, by nodes in the HTM **320**, of a set of probabilities that an input belongs to one or more known or recognized coincidences in the trained HTM **320**, and probabilities that inputs represent a recognized temporal group of coincidences.

(17) When applied in embodiments of the present disclosure, the HTM **320** has two key features: firstly, the HTM **320** is trained based on the first record set **310** to represent a model of the characteristics of the web application **298** during the training mode of operation; and secondly the HTM **320** can determine whether subsequent record sets are recognizable to the HTM **320** and thus bear similarity to the operation of the web application **298** during training mode of operation. It is to be recalled that, during the training mode of operation, the web application **298** is not encumbered by malicious activity and, thus, a failure to recognize subsequent record sets by the HTM **314** in an inference mode of operation is indicative of web application characteristics that are anomalous.

(18) While the HTM **320** has been described, by way of overview, structurally here, it will be appreciated that its implementation can be a logical representation or approximation of such a structure including a mathematical implementation employing, for example, linear algebra and/or parallel processing means for implementation.

(19) The HTM **320** is trained by a HTM trainer **314** which is a hardware, software, firmware or combination component adapted to undertake the training of the HTM **320**. It will be appreciated, on the basis of the above referenced papers and books, that the HTM **320** preferably operates on the basis of a sparse distributed representation (SDR) **312** of data. For example, an SDR can be a binary representation of data comprised of multiple bits in which only a small percentage of the bits are active (i.e. binary 1), and/or equivalent efficient representations of such schemes which may be compressed or the like (e.g. run-length encoded or the minority of bits that are active being somehow listed or otherwise identified). The bits in these representations have semantic meaning and meanings are distributed across the bits. SDR is described in “Sparse Distributed Representations” (Numenta, available from github.com/numenta/nupic/wiki/Sparse-Distributed-Representations and accessed on 29.sup.th March 2017). Further, the principles underlying SDR are also described in “Sparse coding with an overcomplete basis set: A strategy employed by V1?” (Olshausen, B. A., Field, D. J., 1997, Vision Research, 37:3311-3325). Accordingly, the records in the first record set **310** are initially encoded to an SDR by a suitable encoder. Notably, the encoder is configured to set bits in a SDR **312** for a record based on a semantic meaning of the bits and thus the encoder is specifically configured to encode each record in to a SDR **312** based on semantic meaning of some aspect of the record including, for example, one or more of: a content of the record; characteristics of the record such as its length, origin, when it was received, how it was created, what created it etc.; what the record means, what it indicates, what consequence may ensue as a result of an occurrence recorded by the record etc.; and other aspects as will be apparent to those skilled in the art.

(20) Thus, in use, the web application change detector **202** trains the HTM **320** using SDR representation **312** of records received in the first record set **310** for the web application **298** in use during a training mode of operation with a training client **306**. The training time period, during which the web application **298** operates in the training mode of operation and the record collector **201** collects records of characteristics, can be determined based on a degree of recognition by the HTM **320** of new training records in the first record set **310**. That is to say that the HTM Trainer **314** can additionally operate with the HTM **320** to determine a degree of recognition of each record in the first record set **310**. In the early phases of training, a relatively greater proportion of records in the first record set **310** will cause a low degree of recognition by the HTM **320** since the HTM **320** will

be relatively continuous. As training continues, the model of the HTM 320 will improve to reflect the characteristics of the web application 298 and the degree of recognition will increase. The training period can be flexible to expire based on a degree of recognition of the HTM 320 of records in the first record set 310 such that when such a degree of recognition meets a predetermined threshold, then training can be configured to cease or reduce. In one embodiment, the HTM 320 is adapted to provide an anomaly score for records in the first record set 310 reflective of a degree of recognition and it is the anomaly score meeting a predetermined threshold that determines when training should cease or reduce.

(21) Accordingly, following training, the HTM 320 can be said to constitute a model or record of characteristics of the web application 298 during the training time period for which the first record set 310 was received. This model is subsequently used to detect an anomalous state of characteristics of the web application 298 as will be described with respect to FIG. 4.

(22) FIG. 4 is a component diagram illustrating the operation of the web application change detector component 202 to detect a change for a web application 298 in accordance with embodiments of the present disclosure. The client 206 in FIG. 4 is operable to communicate with the web application 298 in a normal or production mode of operation in which there is no assurance, implication or presumption that the computer system 200, the web server 296, the web application 298 and/or the client 206 are operating without the influence, encumbrance or effect of malicious software and/or communication. The period during which the client operates in the normal or production mode of operation can be referred to as an operational time period and is defined as distinct to the training time period. During the operational time period, the record collector 201 generates a second set of records 410 of characteristics of the web application 298 substantially as hereinbefore described with reference to FIG. 3.

(23) Thus, the web application change detector 202 receives a second record set 410 from the record collector 201 relating to the operational time period. Subsequently, an SDR 412 of the records of the second log is generated by an encoder substantially as previously described with respect to the first record set 310. An HTM executer 414 then executes the HTM 320 (now trained by way of the arrangement of FIG. 3) in an inference or recognition mode of operation. In this mode of operation, the HTM 320 determines a degree of recognition of each SDR data item input to it based on its learned model of the use of the web application 298 during the training time period. Based on this degree of recognition the HTM 320 also identifies anomalies as SDR inputs that are not recognized by the trained HTM 320.

(24) The HTM 320, modelling the web application 298 during the training time period, will indicate a strong degree of recognition of SDR for records of the second record set 410 arising from operation of the web application 298 exhibiting characteristics consistent with those during the training period. If, however, anomalies are detected by the HTM 320 such that records from the second record set 410 are not recognized by the HTM 320, such anomalies indicate web application 298 characteristics deviating from those exhibited during the training time period. An anomaly can be identified by the HTM 320 based on a threshold degree of similarity of SDR 412 for records in the second record set 410. Thus, where anomalies are identified by the HTM 320 then a change to the configuration or web page(s) of the web application 298 are identified (depending on the characteristics used to generate the records in the first record set 310 of the training phase).

(25) A change determiner 416 is a software, hardware, firmware or combination component responsive to the HTM executer 414 to determine that a substantive change to characteristics of the web application 298 has been detected. In one embodiment, a substantive change is identified when a degree of recognition (or an anomaly score) generated with reference to the HTM 320 exceeds a predetermined threshold. Depending on the nature of the web application characteristics modelled by the HTM 320, one or more responsive measures can be implemented when a characteristic change is identified. Responsive measures can include one or more of, for example, inter alia: interrupting operation of the web application 298 such as by terminating the application 298, pausing the application 298, preventing new connections to the application 298 or disconnecting existing connections; identifying client components in communication with the web application as potentially compromised by malicious software and/or communications or by disclosure of data; executing an intrusion detection, malware detection, virus removal and/or malware removal process for the web application 298; and effecting a redeployment, reinstallation and/or reconfiguration of the web application 298 which can include software constituting the web application 298 including the web server 296 and/or other software on the computer system 200.

(26) FIG. 5 is a flowchart of a method to detect an anomalous change to a web application 298 executing with a web server 296 in accordance with embodiments of the present disclosure. At 502, the change detector 202 receives the first record set 310 as a first set of records for characteristics of the web application 298 operating in a training mode of operation. At 504 an SDR 312 is generated for each of the records in the first record set 310 to form a training set for the HTM 320. At 506 the HTM 320 is trained based on the training set such that the trained HTM 320 provides a model of the operation of the computer system during the training time period. At 508, the web application change detector 202 receives a second set of records for the computer system 200 as a second record set 410. Each record in the second record set 410 relates to characteristics of the web application 298 during an operational time period in use with a client 206. At 510, an SDR is generated for each of the records in the second record set 410 to form an input set for the trained HTM 320. At 512, the trained HTM 320 is executed based on the input set to determine a degree of recognition of the records of the input set by the HTM 320. At 514, the change detector 202 identifies anomalous characteristics of the web application 298 in the operational time period based on a degree of recognition by the HTM 320, where a degree of recognition below a predetermined threshold identifies an anomaly requiring responsive measures.

(27) An exemplary embodiment of the present disclosure in which configuration characteristics of the web application 298 are used to identify potentially malicious configuration changes will now be described by way of example only. An evaluation of the behavior of a number of different web servers identifies that each web server can exhibit different characteristics based on its implementation, vendor, version, update, or configuration. For example, these differing characteristics can include: HTTP Header Content: each web server can generate HTTP response messages with different header content. Traffic Management: web servers may employ a different traffic management mechanism to manage how responses are communicated to web browsers. For example, such characteristics can determine how network communications are generated, a size of units of communication such as packets, the content of each unit of communication, etc. HTTP Response Messages: different web servers may generate differently structure, formatted or substantively different response messages to web browser requests such as POST requests.

Closing the Web Connections: web servers can determine how a web connection should be closed, for example, whether by using an underlying protocol such as sending “FIN” in TCP/IP or relying on a web browser to close a connection.

(28) How a HTM can be used to model a web application **298** based on configuration characteristics of a web server **296** employed for the application **298** will now be considered with reference to HTTP response messages by way of example only. The table below shows how different web servers can have different response messages.

(29) TABLE-US-00001 Server Exemplary HTTP Response Message Apache HTTP/1.1 200 OK 2.2.4 Date: Thu, 4 Feb. 2016 09:11:47 GMT with PHP Server: Apache/2.2.4 :Win32 DAV/2 mod_ssl/2.2.4 enabled OpenSSL/0.9.8d mod_autoindex_color PHP/5.2.1 Last-Modified: Sat, 11 Nov. 2006 14:13:22 GMT ETag: "7b2fa-1052-456a2480" Accept-Ranges: bytes Content-Length: 4178 Keep-Alive: timeout = 5, max = 99 Connection: Keep-Alive Content-Type: text/css Apache HTTP/1.1 304 Not Modified 2.4.18 Date: Thu, 11 Feb. 2016 10:46:42 GMT with PHP Server: Apache/2.4.18 (Win32) OpenSSL/1.0.2e PHP/7.0.2 enabled Last-Modified: Tue, 6 Oct. 2015 10:03:37 GMT ETag: "5675-52353982e9280" Accept-Ranges: bytes Content-Length: 3145 Connection: Keep-Alive Keep-Alive: timeout = 5, max = 97 Content-Type: text/html Tomcat HTTP/1.1 200 OK 7.0 Server: Apache-Coyote/1.1 Accept-Ranges: bytes ETag: W/"9756-1305217116000" Last-Modified: Thu, 12 May 2011 16:18:36 GMT Content-Type: text/css Content-Length: 9756 Date: Thu, 4 Feb. 2016 09:10:03 GMT

(30) Web application communication traffic from a web server contains characteristics for a web application. Each response message contains a list of attributes and their values. Characteristics include: the order of attributes; the number of attributes; and values of attributes. For example, in the above table, two HTTP response headers from two different web servers are given as examples. An HTM model of web application configuration characteristics can be learned for a web application and the model can be used to detect changes in such characteristics indicative of a configuration change for the web application.

(31) In order to best employ the capability of an HTM it is preferably to adapt characteristic information in an SDR format: 1. Different web applications with potentially different configuration characteristics will have different HTTP response messages. First, response message fields are converted into an attribute sequence with field values as elements of the sequence. For example, the HTTP response from two different web applications executing with different web servers can be turned into the following sequences of characteristics: Apache 2.2.4 Server: Date:value.fwdarw.Server:value.fwdarw.Last-Modified:value.fwdarw.ETag:value.fwdarw.Accept-Ranges:value.fwdarw.Content-Length:value.fwdarw.Keep-Alive:value.fwdarw.Connection:value.fwdarw.Content-Type:value Apache 2.4.18 Server: Date:value.fwdarw.Server:value.fwdarw.Last-Modified:value.fwdarw.ETag:value.fwdarw.Accept-Ranges:value.fwdarw.Content-Length:value.fwdarw.Connection:value.fwdarw.Keep-Alive:value.fwdarw.Content-Type:value Tomcat 7.0 Server: Server:value.fwdarw.Accept-Ranges:value.fwdarw.ETag:value.fwdarw.Last-Modified:value.fwdarw.Content-Type:value.fwdarw.Content-Length:value.fwdarw.Date:value From these examples we can see that, while the names of attributes are standardized, their order is quite different and depends on a type or configuration of a web server used by a web application.

2. These sequences of characteristics form a basis for training the HTM. In one embodiment, the sequences can be regularized using generalization rules. For example, instead of ETag: “7b2fa-1052-456a2480”, a regularized expression can be formulated such as ETag: “{circumflex over ()}([a-z0-9]{5}-[a-z0-9]{4}-[a-z0-9]{8})\$”. Similarly, instead of Date: Thu, 4 Feb. 2016 09:10:03 GMT, a regularized expression can be formulated such as Date: “EEE, dd MMM yyyy HH:mm:ss z”.

(32) Using such an approach the sequences above can be converted to: Apache 2.2.4 Server: Date: “EEE, dd MMM yyyy HH:mm:ss z” .fwdarw.Server: “Apache/2.2.4:Win32 DAV/2 mod_ssl/2.2.4 OpenSSL/0.9.8d mod_autoindex_color PHP/5.2.1” .fwdarw.Last-Modified: “Sat, 11 Nov. 2006 14:13:22 GMT” .fwdarw.ETag: “{circumflex over ()}([a-z-0-9]+-[a-z-0-9]+-[a-z-0-9]+)\$” .fwdarw.Accept-Ranges: “bytes” .fwdarw.Content-Length: “\d+” .fwdarw.Keep-Alive: “timeout=(\d+), max=(\d+)” .fwdarw.Connection: “Keep-Alive” .fwdarw.Content-Type: “{circumflex over ()}([a-z]/[a-z])\$” Apache 2.4.18 Server: Date: “EEE, dd MMM yyyy HH:mm:ss z” .fwdarw.Server: “Apache/2.4.18 (Win32) OpenSSL/1.0.2e PHP/7.0.2” .fwdarw.Last-Modified: “Tue, 6 Oct. 2015 10:03:37 GMT” .fwdarw.ETag: “{circumflex over ()}([a-z-0-9]+-[a-z-0-9]+)\$” .fwdarw.Accept-Ranges: “bytes” .fwdarw.Content-Length: “\d+” .fwdarw.Keep-Alive: “timeout=(\d+), max=(\d+)” .fwdarw.Connection: “Keep-Alive” .fwdarw.Content-Type: “{circumflex over ()}([a-z]/[a-z])\$” Tomcat 7.0 Server: Server: “Apache-Coyote/1.1” .fwdarw.Accept-Ranges: “bytes” .fwdarw.ETag: “{circumflex over ()}(W/^[0-9]+-[0-9]+)\$” .fwdarw.Last-Modified: “Thu, 12 May 2011 16:18:36 GMT” .fwdarw.Content-Type: “{circumflex over ()}([a-z]/[a-z])\$” .fwdarw.Content-Length: “\d+” .fwdarw.Date: “EEE, dd MMM yyyy HH:mm:ss z” 3. For training an HTM, each element of a signature sequence needs to be encoded into SDR. An existing SDR category encoder from an HTM implementation can be used here. For example, a 100 bits SDR can be used with 10% of bits activated to encode each element of a sequence, with bits having semantic significance in the context of the characteristics in a signature sequence. The three examples in the previous step can be encoded as shown in the following tables:

[illegible]

above described example embodiments, the disclosure is not limited thereto and that there are many possible variations and modifications which fall within the scope of the claims.

(54) The scope of the present disclosure includes any novel features or combination of features disclosed herein. The applicant hereby gives notice that new claims may be formulated to such features or combination of features during prosecution of this application or of any such further applications derived therefrom. In particular, with reference to the appended claims, features from dependent claims may be combined with those of the independent claims and features from respective independent claims may be combined in any appropriate manner and not merely in the specific combinations enumerated in the claims.

Claims

1. A computer implemented method to detect an anomalous change to a configuration of a web application, the web application executing with a web server, the method comprising: receiving a first set of distinct records for the web application operating in a training mode of operation during which the web application is isolated from a wide area network, each record in the first set of distinct records including characteristics of a content of a web page the web application during a training time period; generating a sparse distributed representation of the first set of distinct records to form a training set for a hierarchical temporal memory (HTM); training the HTM based on the training set in order that the trained HTM provides a model of operation of the web application in the training mode of operation, wherein the HTM evaluates an anomaly score for the records in the first set of distinct records and the HTM is trained until the anomaly score meets a predetermined threshold degree of anomaly; receiving a second set of distinct records for the web application, each record in the second set of distinct records including characteristics of the web application during an operational time period, the operational time period being distinct from the training time period; generating a sparse distributed representation of the second set of distinct records to form an input set for the trained HTM; executing the trained HTM based on the input set to determine a degree of recognition of the records of the input set; responsive to a determination that a degree of recognition of one or more records of the input set is below a threshold degree, identifying an anomalous change to the configuration of the web page; and in response to the identification of the anomalous change to the web page, causing a responsive measure to the anomalous change to be implemented that includes one or more of the following actions: interrupting operation of the web application; identifying client components in communication with the web application as potentially compromised; executing at least one of an intrusion detection, malware detection, virus removal, or a malware removal process for the web application; or effecting at least one of a redeployment, a reinstallation, or a reconfiguration of the web application.

2. The method of claim 1, wherein the characteristics of the web page include records corresponding to hypertext markup language (HTML) tags in the web page.

3. A computer system comprising: a processor and memory storing computer program code for detecting an anomalous change to a configuration of a web application, the web application executing with a web server, by: receiving a first set of distinct records for the web application operating in a training mode of operation during which the web application is isolated from a wide area network, each record in the first set of distinct records including characteristics of a content of a web page for the web application during a training time period; generating a sparse distributed representation of the first set of distinct records to form a training set for a hierarchical temporal memory (HTM); training the HTM based on the training set in order that the trained HTM provides a model of operation of the web application in the training mode of operation, wherein the HTM evaluates an anomaly score for the records in the first set of distinct records and the HTM is trained until the anomaly score meets a predetermined threshold degree of anomaly; receiving a second set of records for the web application operating in a production mode of operation, each record in the second set of records including characteristics of content of the web page during an operational time period, the operational time period being distinct from the training time period and the production mode of operation being distinct from the training mode of operation; generating a sparse distributed representation of the second set of distinct records to form an input set for the trained HTM; executing the trained HTM based on the input set to determine a degree of recognition of the records of the input set; responsive to a determination that a degree of recognition of one or more records of the input set is below a threshold degree, identifying an anomalous change to the web page; and responsive to the identification of the anomalous change to the web page, causing a responsive measure to the anomalous change to be implemented that includes one or more of the following actions: interrupting operation of the web application; identifying client components in communication with the web application as potentially compromised; executing at least one of an intrusion detection, malware detection, virus removal, or a malware removal process for the web application; or effecting at least one of a redeployment, a reinstallation, or a reconfiguration of the web application.

4. A non-transitory computer-readable storage element comprising computer program code to, when loaded into a computer system and executed thereon, cause the computer system to perform the method as claimed in claim 1.
